



UNIVERSITY OF SCIENCE, VNU-HCM

Faculty of Information Technology

REPORT

CS486 – Introduction to Database System

Project: Campus Space Management System

Lecturer: Le Thi Nhan, PhD.

Teaching Assistants: Nguyen Ngoc Toan, MSc.; Nguyen Ngoc Minh Chau, MSc.

Lab Instructors: Nguyen Ngoc Toan, MSc.; Nguyen Ngoc Minh Chau, MSc.

Group Members:

Vo Minh Dang	23125002
Truong Gia Bao	23125052
Vo Nguyen Minh Triet	24125082

Ho Chi Minh City, July 2026

Contents

Abstract	2
1 Project Overview	2
2 LLM Model Used	2
3 Agent Improvement Process	2
4 Business Requirement Analysis	3
5 Conceptual Database Design	4
6 Logical Database Design	4
7 Database Design Validation	5
8 Database Implementation	5
8.1 Main Tables	6
8.2 Triggers Added by the Group	6
8.2.1 Booking Request Trigger	6
8.2.2 Approval Trigger	6
8.2.3 Usage Session Trigger	7
8.2.4 Maintenance Trigger	7
9 Sample Data Preparation	7
10 Query Design	8
11 Evaluation of Final Outputs	9
12 Limitations	9
13 Conclusion	9

Abstract

This report presents the Phase 1 database design and implementation for the Campus Space Management System. The system is designed for the School of Computer Science to manage shared campus spaces, including classrooms, computer laboratories, auditoriums, meeting rooms, project laboratories, and student workspaces.

The report explains the business requirements, conceptual ERD, logical schema, validation, SQL implementation, sample data, and query design. It also describes how the group used an AI agent, evaluated its output, and improved it through testing. The final database uses tables, keys, constraints, and triggers to protect important business rules such as preventing overlapping approved bookings, blocking unavailable spaces, and checking authorized staff actions.

1 Project Overview

The School of Computer Science currently manages shared spaces manually. Users contact staff by email, phone, or in person to request rooms. Facility staff then check calendars or spreadsheets to decide whether a room is available. This process is slow and can lead to mistakes when the number of classes, seminars, workshops, and student activities increases.

The Campus Space Management System solves this problem by storing booking, approval, usage, maintenance, and facility data in one database. The system helps staff manage spaces fairly, avoid conflicting bookings, prevent the use of unavailable spaces, and keep historical records.

The main goals of the database are:

- Store university users and their roles.
- Store managed spaces and their facilities.
- Allow users to submit booking requests.
- Allow facility staff or managers to approve or reject bookings.
- Record actual check-in and completion information.
- Track maintenance issues and assigned staff.
- Support useful reports about bookings, maintenance, and space utilization.

2 LLM Model Used

The group used GitHub Copilot as the AI assistant during the project. The model helped analyze requirements, draft database design documents, generate SQL scripts, prepare sample data, and design SQL queries.

The group did not use the model output without review. Each task was checked against the business requirements and tested when possible. SQL scripts were executed with Microsoft SQL Server LocalDB to find errors and confirm that constraints, triggers, sample data, and queries worked correctly.

3 Agent Improvement Process

The group improved the agent in several steps.

First, the agent read the project requirements and produced the business requirement analysis. The group checked whether the actors, entities, attributes, relationships, and business rules matched the project description.

Second, the agent generated the ERD and logical schema. The group reviewed whether every main entity was included: users, spaces, facilities, booking requests, approvals, usage sessions, and maintenance records. The group also checked keys, foreign keys, and cardinalities.

Third, the agent generated the SQL DDL for Task 5. The first version had the main tables and many basic constraints, but some rules needed stronger checks. The group asked the agent to review Task 5 against the requirements. The review found gaps such as missing role checks for active staff, weak workflow checks, and missing validation for some invalid cases.

Fourth, the agent improved the implementation by adding triggers. The group evaluated the triggers by running normal sample data and exceptional test cases. When an error appeared in Task 6 because identity values caused booking IDs to mismatch approval records, the agent fixed the sample data by using fixed booking IDs with `IDENTITY_INSERT`.

Finally, the agent expanded Task 7 from 7 queries to 15 queries. The group executed the queries, exported results from `query1.csv` to `query15.csv`, and checked whether the outputs matched the sample data.

4 Business Requirement Analysis

The system has six actor types:

- Student
- Lecturer
- Teaching assistant
- Facility staff
- Department administrator
- Facility manager

The main entities are:

- User
- Space
- Facility
- Booking Request
- Approval
- Usage Session
- Maintenance Record

The main business rules are:

- Every user must have a university account.

- User roles must be controlled.
- A space must have a unique space code.
- A space can be available, in use, under maintenance, temporarily closed, or retired.
- A booking request must select one requester and one space.
- The requested end time must be later than the requested start time.
- Expected participants must be positive and should not exceed space capacity.
- The same space cannot have two approved bookings with overlapping time periods.
- A space under maintenance, temporarily closed, or retired cannot be booked.
- Only active facility staff or facility managers can approve or reject bookings.
- Only approved bookings can be checked in.
- Maintenance assignments must be given to active facility staff or facility managers.

5 Conceptual Database Design

The conceptual design uses one general **USER** entity for all account holders. The **role** attribute controls what a user can do. For example, facility staff and facility managers can approve bookings, while students and lecturers can submit booking requests.

The **SPACE** entity stores physical spaces. Each space can have many facilities and many booking requests over time.

The **FACILITY** entity stores available equipment in each space, such as projectors, whiteboards, microphones, computers, livestreaming equipment, and air conditioners.

The **BOOKING_REQUEST** entity stores room reservation requests. Each request is made by one user and selects one space.

The **APPROVAL** entity stores approval or rejection decisions. Each approval is linked to one booking request and one staff member.

The **USAGE_SESSION** entity stores actual check-in and completion details. It records actual start time, check-in staff, initial condition, actual end time, final condition, and usage notes.

The **MAINTENANCE_RECORD** entity stores reported space problems and maintenance progress.

6 Logical Database Design

The ERD was converted into seven relations:

- **users**
- **spaces**
- **facilities**
- **booking_requests**
- **approvals**

- `usage_sessions`
- `maintenance_records`

The main keys are:

- `users.user_id` is the primary key for users.
- `spaces.space_code` is the primary key for spaces.
- `facilities.facility_id` is the primary key for facilities.
- `booking_requests.booking_id` is the primary key for bookings.
- `approvals.booking_id` is both primary key and foreign key.
- `usage_sessions.booking_id` is both primary key and foreign key.
- `maintenance_records.maintenance_id` is the primary key for maintenance records.

Important foreign keys include:

- `booking_requests.requester_id` references `users.user_id`.
- `booking_requests.space_code` references `spaces.space_code`.
- `facilities.space_code` references `spaces.space_code`.
- `approvals.staff_id` references `users.user_id`.
- `usage_sessions.checked_in_by` references `users.user_id`.
- `maintenance_records.reporter_id` and `assigned_staff_id` reference `users.user_id`.

7 Database Design Validation

The schema correctly represents the ERD. Each entity from the conceptual design is mapped to one table. One-to-many relationships are represented with foreign keys. One-to-zero-or-one relationships, such as booking to approval and booking to usage session, are represented by using `booking_id` as the primary key in the child table.

The schema satisfies normal relational rules using primary keys, foreign keys, unique constraints, not-null constraints, and check constraints. Examples include unique user emails, valid user roles, valid space statuses, positive capacity, valid booking purposes, valid booking statuses, and valid maintenance statuses.

Some rules cannot be fully enforced by simple `CHECK` constraints because they need data from other rows or tables. These rules are handled by triggers in Task 5. Examples include role checking, active account checking, capacity checking against the selected space, unavailable space checking, and approved-booking overlap checking.

8 Database Implementation

The database was implemented for Microsoft SQL Server. The implementation includes all required tables, keys, constraints, default values, indexes, a helper function, and triggers.

The helper function `fn_time_ranges_overlap` checks whether two time ranges overlap. It is used by the booking trigger to prevent conflicting approved bookings.

8.1 Main Tables

The implementation creates these tables:

- `users`
- `spaces`
- `facilities`
- `booking_requests`
- `approvals`
- `usage_sessions`
- `maintenance_records`

8.2 Triggers Added by the Group

The group added triggers to enforce business rules that depend on related records.

8.2.1 Booking Request Trigger

The trigger `trg_booking_requests_validate` validates booking requests after insert or update.

It throws these errors:

- **51001:** The requester is not active.
- **51002:** The selected space is under maintenance, temporarily closed, or retired.
- **51003:** Expected participants exceed the selected space capacity.
- **51004:** The booking overlaps another approved or checked-in booking for the same space.
- **51009:** Core booking information is changed after the booking has already been approved, checked in, or completed.

8.2.2 Approval Trigger

The trigger `trg_approvals_validate_and_sync` validates approval records.

It throws this error:

- **51005:** The approver is not an active facility staff member or facility manager.

The trigger also updates `booking_requests.booking_status`. If the decision is approved, the booking becomes approved. If the decision is rejected, the booking becomes rejected.

8.2.3 Usage Session Trigger

The trigger `trg_usage_sessions_validate_and_sync` validates usage sessions.

It throws these errors:

- **51006:** A usage session is created for a booking that is not approved.
- **51008:** The check-in user is not an active facility staff member or facility manager.

The trigger also updates the booking status. If `actual_end_time` is null, the booking becomes checked in. If `actual_end_time` is recorded, the booking becomes completed.

8.2.4 Maintenance Trigger

The trigger `trg_maintenance_records_validate_and_sync` validates maintenance records.

It throws this error:

- **51007:** The assigned maintenance staff member is not an active facility staff member or facility manager.

The trigger also changes a reported maintenance record with assigned staff to assigned. It also marks the related space as under maintenance when the maintenance status is reported, assigned, or in progress.

9 Sample Data Preparation

The sample data supports both normal operations and exceptional cases.

Normal sample data includes:

- Active, inactive, and suspended users.
- All main user roles.
- Available, temporarily closed, and retired spaces.
- Facilities such as projectors, whiteboards, computers, microphones, livestreaming equipment, and air conditioners.
- Booking requests with pending, approved, rejected, cancelled, checked-in, completed, and no-show statuses.
- Approval decisions by active facility staff and facility managers.
- Usage sessions for checked-in and completed bookings.
- Maintenance records with assigned, in-progress, completed, and cancelled examples.

The script also contains exceptional test cases using `TRY/CATCH`. These cases are expected to fail and prove that the triggers work correctly.

The tested error cases are:

- Inactive user tries to submit a booking request.

- A user tries to book a temporarily closed, retired, or under-maintenance space.
- Expected participants exceed the space capacity.
- A booking approval creates an overlapping approved booking.
- A suspended staff member tries to approve a booking.
- A usage session is created for a pending booking.
- A suspended staff member is assigned to maintenance.
- A student tries to check in a booking.
- Core booking information is edited after approval.

10 Query Design

The group designed 15 SQL queries. Each query includes a business question, target users, explanation, and SQL statement.

The queries answer these business questions:

1. Which booking requests are scheduled or recently processed, and what is their approval and usage status?
2. Which spaces are available for a requested time period and have enough capacity?
3. Which spaces are used most heavily by approved, checked-in, or completed bookings?
4. What unresolved maintenance work exists, and who is responsible for it?
5. Which booking requests still need staff attention or were rejected, and why?
6. Which approved bookings have no usage session recorded yet?
7. What is the complete facility inventory for each space, including maintenance exposure?
8. How many bookings exist by day, status, and purpose?
9. Which requester roles have the most cancelled, rejected, or no-show bookings?
10. Which approved bookings are past their scheduled end time but still have no usage session?
11. How long did resolved maintenance cases take to complete or cancel?
12. Which spaces are missing common facilities needed for teaching or events?
13. How much approval work has each facility staff member or manager handled?
14. Which active maintenance records may affect pending or approved future bookings in the same space?
15. What is utilization by month, space type, and purpose of use?

The group exported the results to `query1.csv` through `query15.csv`. The outputs were checked against the sample data. The results matched the expected booking statuses, approval records, usage sessions, maintenance records, and aggregate calculations.

11 Evaluation of Final Outputs

The final repository contains the required files:

- AGENT.md
- SKILL.md
- outputs/01-business-req-analysis-G10.md
- outputs/02-erd-design-G10.md
- outputs/03-logical-design-G10.md
- outputs/04-design-validation-G10.md
- outputs/05-db-definition-G10.sql
- outputs/06-sample-data-G10.sql
- outputs/07-query-design-G10.sql

The SQL implementation and sample data were executed successfully on SQL Server LocalDB. The query script was also executed successfully. This shows that the database design is usable and that the main business rules are enforced.

12 Limitations

The current database focuses on Phase 1 requirements. Some parts could be improved in a later phase:

- The system does not include a user interface.
- Some workflow rules may need stored procedures in a production system.
- Notification features are not included.
- More sample data would be useful for larger performance testing.
- More detailed audit logs could be added for status changes.

13 Conclusion

The Campus Space Management System database satisfies the main Phase 1 requirements. It stores users, spaces, facilities, booking requests, approvals, usage sessions, and maintenance records. It also uses constraints and triggers to protect important business rules.

The group improved the agent output by reviewing each task, testing the SQL scripts, finding weak points, and refining the implementation. The final database can prevent invalid bookings, block unavailable spaces, check authorized staff actions, record usage history, and support useful reports for staff and managers.